CookShare

Ecosistema Multiplataforma para la Gestión de Recetas y Compra Inteligente

https://github.com/NoeliaFreire04/proyecto-dam

Nombre Alumno/a: **Noelia Freire Amarelo**

Curso: ***2º DAM***

Módulo: ***Proyecto Final Ciclo***

Proyecto 1

Contenido

Contenido 2
1. Introducción 3
2. Objetivos 3
3. Situación previa 4
4. Tecnologías empleadas 4
Frontend 4
Backend 4
Infraestructura e Innovación 4
5. Solución propuesta 5
5.1. Definición de alcance 5
5.3. Presupuesto 6
6. Planificación del proxecto 6
6.1. Modelo de desarrollo 6
6.2. Diagrama de Gantt 7
6.3. Descripción do modelo de desenvolvemento de software a implementar. 7
6.4. Análise do proxecto: 8
○ Diagrama de casos de usos. 8
○ Universo de discurso da base de datos. 8
○ Diagrama de Entidad-Relación. 8
6.5. Deseño do proxecto: 9
○ Modelo relacional da base de datos. 9
○ Diagrama de clases. 10
○ Diagramas de flujo de cada caso de uso. 10
○ Mockups de la interfaz. 11
6.6. Presuposto completo (Hardware software e recursos humanos) 12
6.6.1. Recursos Humanos 12
6.6.2. Hardware 12
6.6.3. Software y servicios cloud 12
6.6.4. Total estimado 13
7. Desarrollo y ejecución 14
7.1. Diagrama de despliegue 14
7.3. Detalles técnicos resueltos en el desarrollo 15
8. Conclusións y reflexiones 15
9. Bibliografía y Webgrafía 15
10. Anexo I – Manual técnico de instalación o puesta en marcha 15
11. Anexo II – Documentación de uso (manuales usuario) 15
12. Anexo III – Otra documentación 15

1. Introducción

El proyecto CookShare consiste en el desarrollo de una plataforma multiplataforma (móvil y servidor) diseñada para transformar la gestión de la alimentación doméstica. La aplicación combina las funciones de un recetario personal con herramientas avanzadas de productividad, como la generación de listas de la compra a partir de recetas seleccionadas y el escalado inteligente de ingredientes según el número de comensales.

Como factor innovador, la app incorpora un módulo de Inteligencia Artificial que permite la extracción automática de recetas a partir de contenido audiovisual (Video-to-Recipe), facilitando la digitalización de procesos que, de forma tradicional, resultan manuales y tediosos.

El proyecto ha sido desarrollado siguiendo un modelo de desarrollo ágil e incremental, con una estimación total de 150 horas de trabajo distribuidas entre investigación, backend, frontend y documentación.

2. Objetivos

- **Gestión Inteligente:**
  - Implementar un algoritmo de cálculo dinámico para el escalado de ingredientes proporcional al número de comensales seleccionado, aplicando la fórmula: Cantidad_nueva = (Cantidad_base / Comensales_base) × Comensales_deseados.
- **Automatización de la Lista de la Compra:**
  - Desarrollar un sistema que transforme los ingredientes de una receta en artículos de una lista de la compra, cruzándolos con un inventario básico gestionado manualmente por el usuario.
- **Plataforma Social Básica:**
  - Crear un sistema donde los usuarios puedan registrarse, publicar sus propias recetas (públicas o privadas) y guardar las de otros miembros de la comunidad en una sección de Favoritos.
- **Robustez Técnica:**
  - Diseñar un backend con Java 21 y Spring Boot bajo una arquitectura multicapa que garantice la seguridad (JWT) y persistencia de un modelo de datos con más de 10 relaciones.
- **Innovación mediante IA:**
  - Implementar el módulo Video-a-Receta con la API de Google Gemini para procesar archivos de vídeo y extraer automáticamente el título, los ingredientes y los pasos de elaboración de una receta.

3. Situación previa

Actualmente existe una desconexión importante entre el momento de inspiración culinaria (ver una receta en un vídeo, en redes sociales o en un blog) y la logística práctica de llevarla a cabo: calcular qué cantidad de cada ingrediente se necesita para más o menos personas, y saber qué hay que comprar teniendo en cuenta lo que ya se tiene en casa.

Las soluciones existentes en el mercado (Tasty, Yummly, Paprika) ofrecen recetarios completos, pero ninguna integra de forma fluida el escalado de ingredientes con una lista de la compra conectada a un inventario personal. Por otro lado, las apps de lista de la compra (OurGroceries, AnyList) no tienen integración con recetas. CookShare aborda esta brecha con una solución unificada.

Respecto a la extracción de recetas desde vídeo, actualmente el proceso es completamente manual: el usuario debe pausar el vídeo, tomar notas y transcribir los pasos. El módulo de IA de CookShare automatiza este proceso por completo..

4. Tecnologías empleadas

Frontend

- Flutter (Dart): framework multiplataforma de Google que genera interfaces nativas y fluidas para Android e iOS desde un único código base, empleando Clean Architecture por capas para garantizar la mantenibilidad del código.

Backend

- Java 21 con Spring Boot 3+: estándar de la industria para APIs REST robustas y escalables.
- Spring Data JPA (Hibernate): abstracción de la capa de persistencia, mapeando las entidades del modelo relacional a objetos Java.
- Spring Security con JWT: autenticación stateless mediante JSON Web Tokens, garantizando que cada recurso sea accesible únicamente por su propietario.
- Swagger / OpenAPI 3: documentación interactiva de la API, facilitando el testeo y la integración con el cliente móvil.

Infraestructura e Innovación

- Base de Datos: MySQL, sistema relacional que garantiza la integridad referencial en un modelo de datos con más de 10 relaciones.
- IA: Google Gemini API, para el procesamiento de vídeo y la extracción de información estructurada.
- Control de versiones: Git con GitHub, usando ramas por funcionalidad y commits diarios.

5. Solución propuesta

Se propone el desarrollo de CookShare, una aplicación multiplataforma construida con Flutter y un backend robusto en Spring Boot. A diferencia de otras alternativas de mercado, CookShare no solo funciona como un recetario, sino como un asistente logístico que cubre todo el flujo alimentario: desde la inspiración en el feed público hasta la generación de la lista de la compra personalizada.

5.1. Definición de alcance

El alcance completo del proyecto, está disponible en el repositorio: Definición de alcance.

5.3. Presupuesto

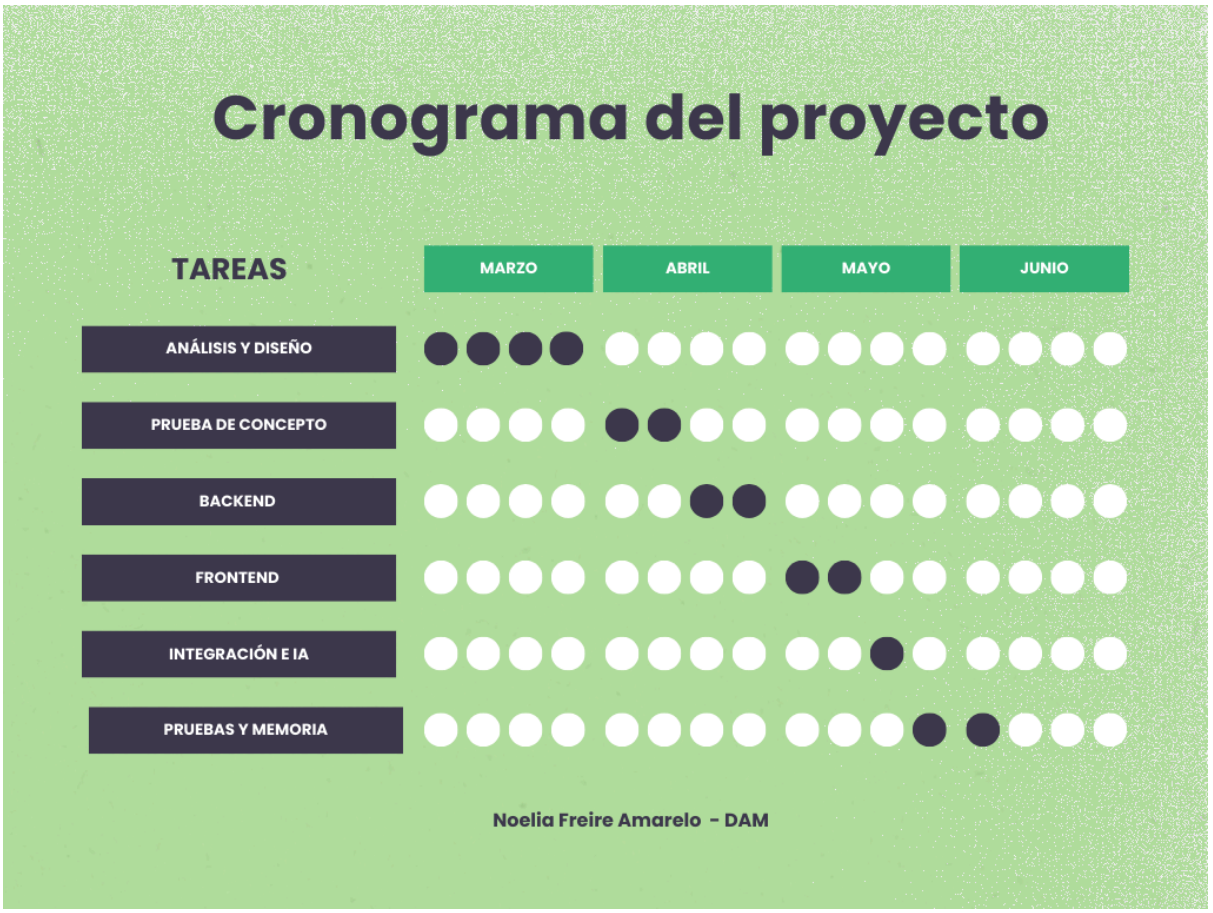
Categoría	Descripción	Estimación
Recursos Humanos	150 horas de desarrollo (Investigación, Backend, Frontend y Documentación) a 18 €/h.	2.700 €
Hardware	Equipo de desarrollo personal (amortización parcial) y smartphone.	200 €
Software y CloudHosting	Base, dominio y API de IA (Tier gratuito/desarrollo).	100 €
TOTAL ESTIMADO		**3.000 €**

6. Planificación del proxecto

6.1. Modelo de desarrollo

Se adopta un modelo de desarrollo ágil e incremental, dividiendo el proyecto en sprints semanales con entregas validadas. Este modelo permite adaptar los requisitos ante imprevistos y asegura que, en cada hito, exista una versión funcional del software.

6.2. Diagrama de Gantt



6.3. Descripción do modelo de desenvolvemento de software a implementar.

El modelo de desarrollo adoptado para CookShare es el modelo incremental con prácticas ágiles.

Este modelo divide el desarrollo en cuatro incrementos, cada uno alineado con una entrega formal del calendario del centro. Al finalizar cada incremento el sistema es funcional para el subconjunto de casos de uso implementados hasta ese momento, lo que garantiza que siempre existe una versión demostrable del software.

Se ha descartado el modelo en cascada porque exige tener todos los requisitos cerrados desde el inicio, y los requisitos de CookShare han evolucionado a raíz del feedback del tutor. El modelo iterativo puro tampoco encaja porque carece de la estructura de entregables formales que exige el calendario del centro.

La componente ágil se concreta en sprints semanales, un mínimo de dos commits diarios en GitHub usando la estrategia Feature Branch Workflow, y una definición de hecho clara: un

caso de uso se considera terminado cuando el endpoint del backend responde correctamente y la pantalla Flutter lo consume sin errores.

6.4. Análise do proxecto:

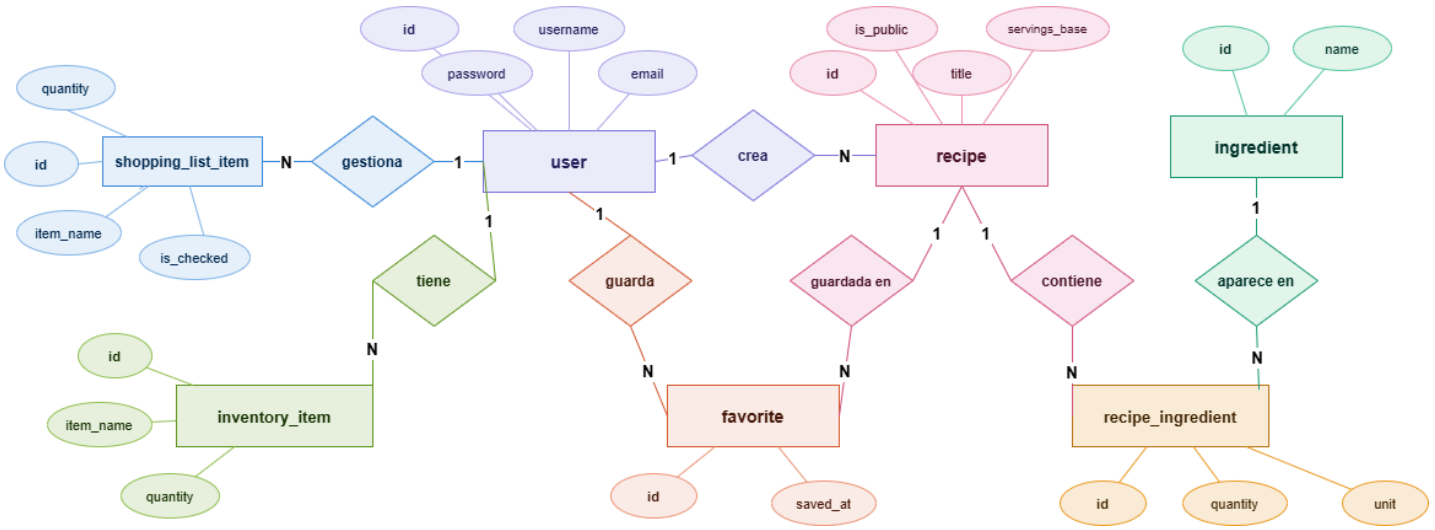
- Diagrama de casos de usos.

  Navegar a documento de Casos de uso

- Universo de discurso da base de datos.

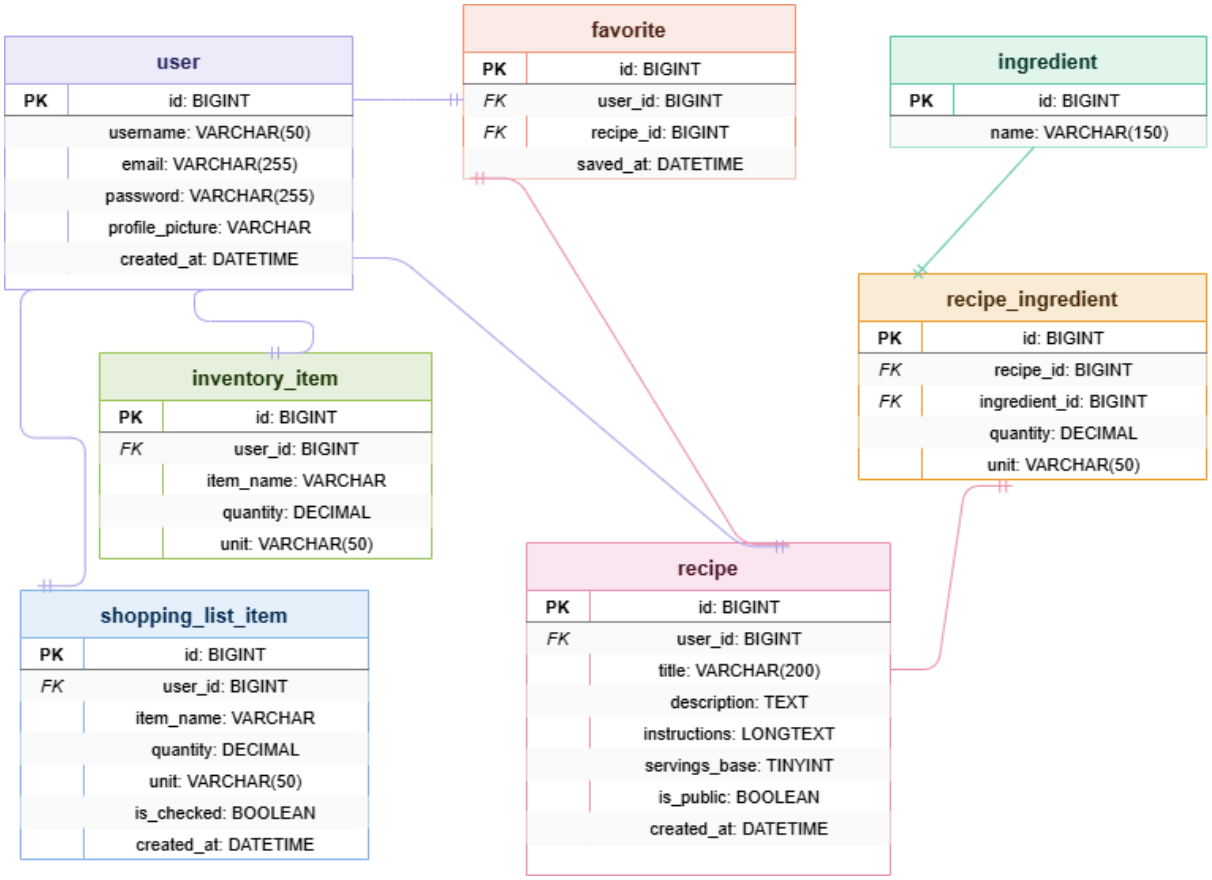
  Navegar a documento de Universo de discurso de base de datos

- Diagrama de Entidad-Relación.

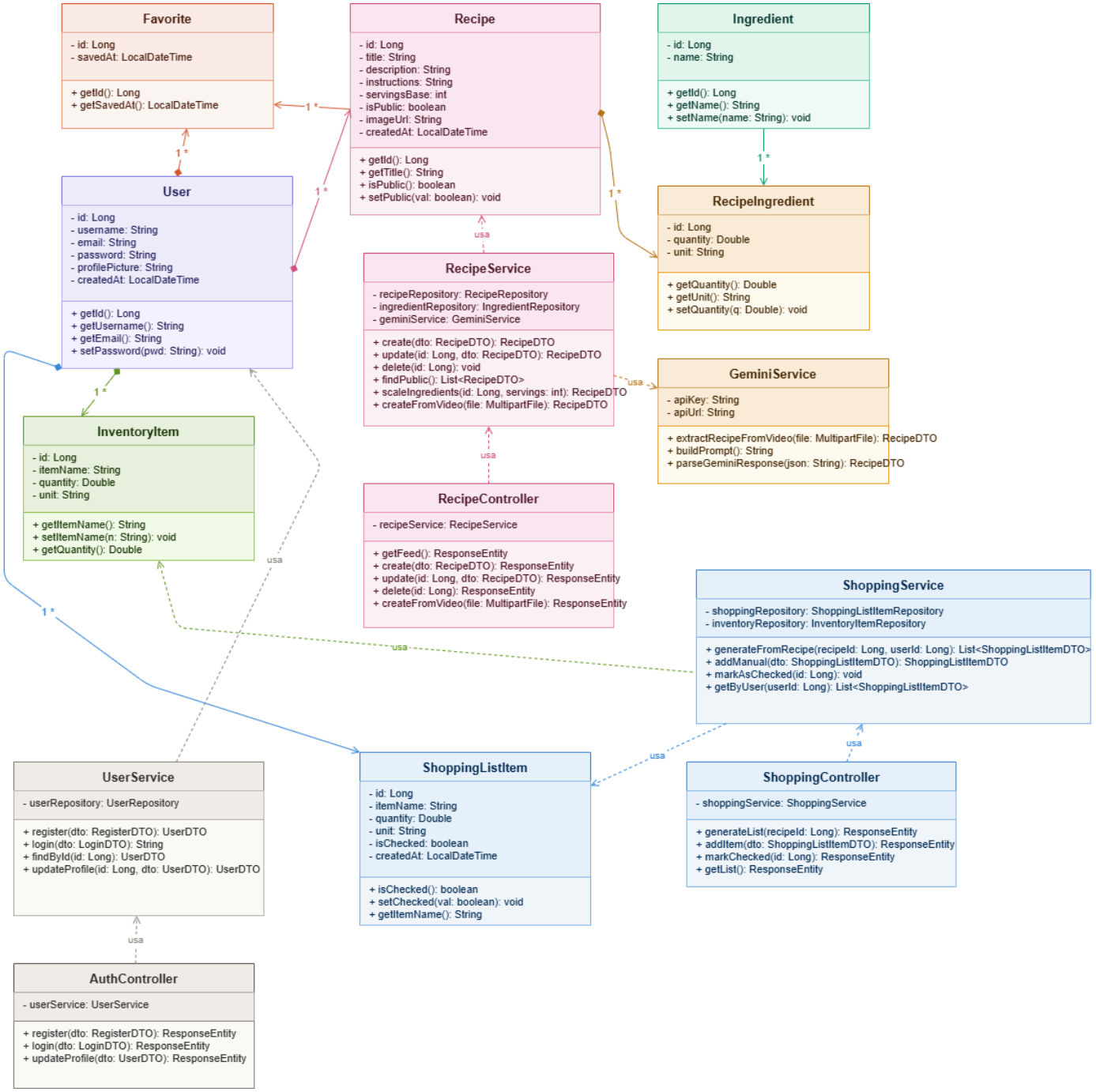


6.5. Deseño do proxecto:

- Modelo relacional da base de datos.



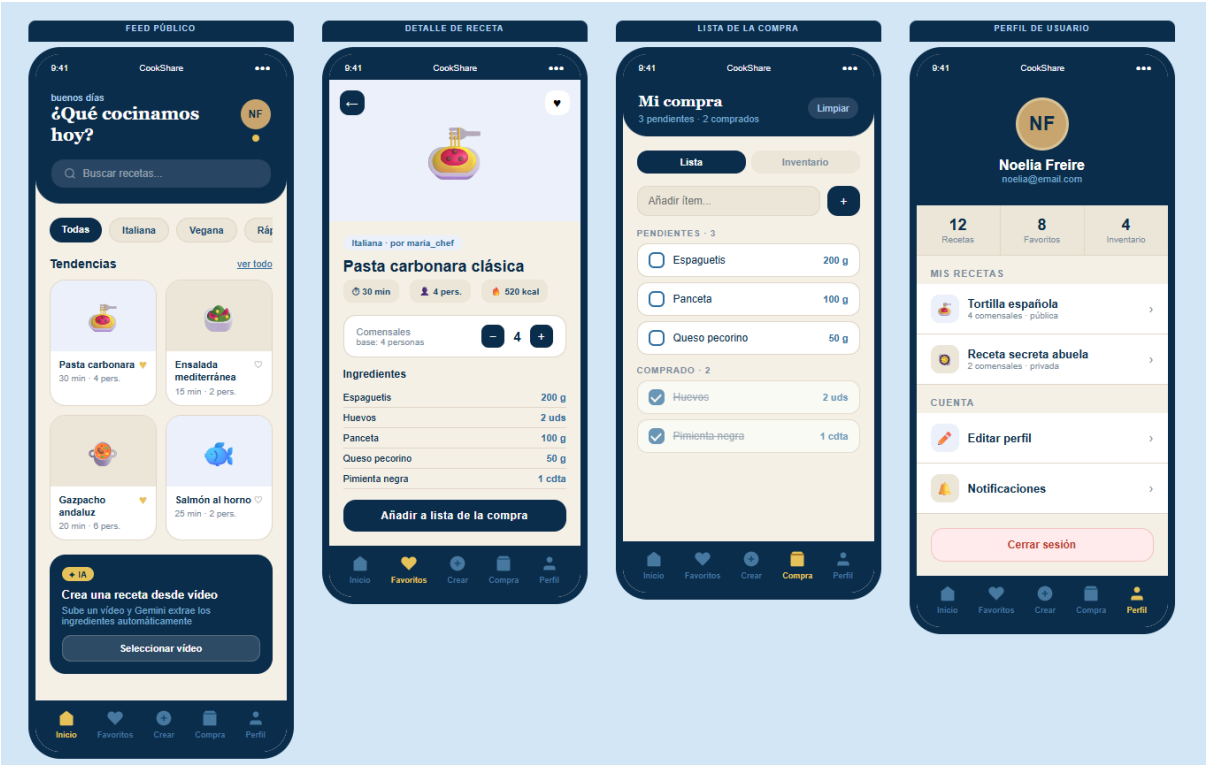
- Diagrama de clases.

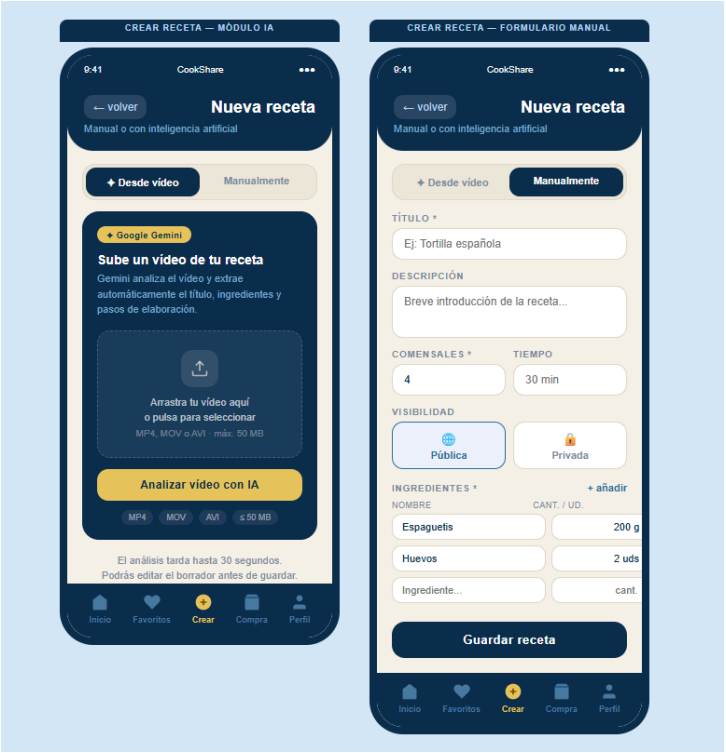


- Diagramas de flujo de cada caso de uso.

Navegar a anexo de [Diagramas de Flujo]

- Mockups de la interfaz.





6.6. Presuposto completo (Hardware software e recursos humanos)

6.6.1. Recursos Humanos

Tarea	Horas	Tarifa/h	Subtotal
Investigación y análisis	15	18,00 €	**270,00 €**
Backend — Spring Boot	55	18,00 €	**990,00 €**
Frontend — Flutter	45	18,00 €	**810,00 €**
Módulo IA — Gemini	15	18,00 €	**270,00 €**
Testing y QA	10	18,00 €	**180,00 €**
Documentación técnica	10	18,00 €	**180,00 €**
TOTAL RECURSOS HUMANOS	**2700,00 €**		

6.6.2. Hardware

Elemento	Justificación	Coste	Importe imputado
Equipo de desarrollo (portátil)	Amortización parcial (vida útil 4 años, uso proyecto 4 meses)	150,00 €	**150,00 €**
Smartphone Android (dispositivo de pruebas)	Amortización parcial para pruebas de la app Flutter	50,00 €	**50,00 €**
TOTAL HARDWARE	**200,00 €**		

6.6.3. Software y servicios cloud

Herramienta / Servicio	Uso	Coste licencia	Importe imputado
IntelliJ IDEA Community	IDE para el desarrollo del backend Spring Boot	0,00 €	**0,00 €**
Android Studio / VS Code	IDE para el desarrollo Flutter	0,00 €	**0,00 €**
MySQL Community Server	Base de datos relacional del proyecto	0,00 €	**0,00 €**

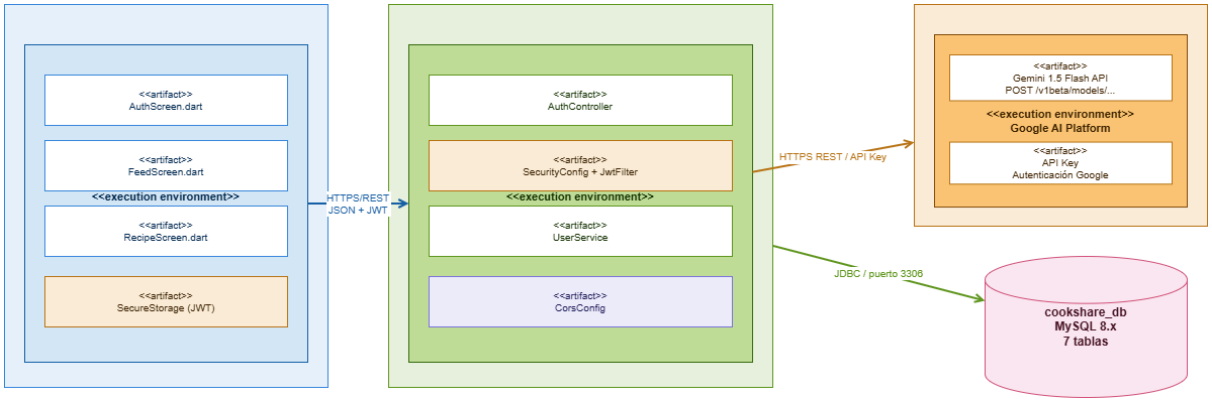
Git + GitHub	Control de versiones y repositorio público	0,00 €	**0,00 €**
Google Gemini API (free tier)	Procesamiento de vídeo y extracción de recetas con IA	0,00 €	**0,00 €**
Servidor de desarrollo local	Entorno local (localhost) — sin costes de hosting en esta fase	0,00 €	**0,00 €**
Draw.io / Figma (free tier)	Herramientas de diseño y diagramación	0,00 €	**0,00 €**
Dominio (previsión fase producción)	Dominio .com o .es para futura puesta en producción (estimación)	15,00 €	**15,00 €**
Hosting cloud (previsión fase producción)	Servidor VPS o PaaS para despliegue del backend Spring Boot	85,00 €	**85,00 €**
TOTAL SOFTWARE Y CLOUD	**100,00 €**		

6.6.4. Total estimado

#	Categoría	Importe
1	**Recursos Humanos — 150 horas × 18,00 €/h**	**2.700,00 €**
2	**Hardware — amortización equipo y smartphone**	**200,00 €**
3	**Software y Cloud — licencias y hosting (estimación producción)**	**100,00 €**
	SUBTOTAL	**3.000,00 €**
	IVA (21%)	**630,00 €**
TOTAL ESTIMADO (IVA incluido)		**3.630,00 €**

7. Desarrollo y ejecución

7.1. Diagrama de despliegue



7.2. Descripción general del funcionamiento

Al iniciar la aplicación, el usuario accede a la pantalla de autenticación, donde se presenta un formulario de inicio de sesión con los campos de correo electrónico y contraseña. En caso de no disponer de una cuenta, el usuario puede navegar al formulario de registro, donde deberá introducir su nombre de usuario, correo electrónico y contraseña. Una vez completados los datos correctamente y confirmada la autenticación, la aplicación redirige al usuario a la pantalla principal de la aplicación.

La comunicación entre el cliente Flutter y el servidor Spring Boot se realiza mediante peticiones HTTP en formato JSON. Cuando un usuario se registra o inicia sesión, Flutter envía una petición POST al endpoint correspondiente con los datos del formulario. El backend procesa la solicitud y responde con un objeto JSON que contiene el token JWT, el cual es almacenado de forma segura en el dispositivo mediante FlutterSecureStorage.

En cuanto a la seguridad de las contraseñas, estas nunca se almacenan en texto plano en la base de datos. En su lugar, se aplica el algoritmo BCrypt, que genera un hash irreversible de la contraseña. Al iniciar sesión, BCrypt compara la contraseña introducida con el hash almacenado sin necesidad de desencriptarlo.

Una vez autenticado correctamente, el servidor genera un token JWT que contiene el correo electrónico del usuario y una fecha de expiración de 24 horas. Este token se incluye en la cabecera Authorization de cada petición posterior a la API. El componente JwtFilter intercepta cada petición, verifica la validez del token y permite o deniega el acceso al recurso solicitado.

7.3. Detalles técnicos resueltos en el desarrollo

Durante el desarrollo surgieron diversas dificultades técnicas que requirieron investigación y adaptación de las soluciones.

La primera dificultad encontrada fue que IntelliJ IDEA Community Edition no activaba automáticamente el procesador de anotaciones de Lombok, lo que generaba errores de compilación al no reconocer anotaciones como @Data o @Builder. Se solucionó habilitando manualmente el procesamiento de anotaciones desde la configuración del compilador y creando constructores en algunos casos donde el procesamiento de anotaciones no tuvo resultado.

Posteriormente, fue necesario prestar especial atención a la configuración del filtro de autenticación JWT. Spring Security requiere que los filtros personalizados extiendan OncePerRequestFilter para garantizar que se ejecutan exactamente una vez por petición. El JwtFilter implementado intercepta cada solicitud entrante, extrae y valida el token JWT de la cabecera Authorization y, si es válido, registra al usuario como autenticado en el contexto de seguridad de Spring, permitiendo el acceso a los recursos protegidos.

Finalmente, al conectar la aplicación Flutter web con el backend, el navegador bloqueaba las peticiones HTTP por la política de seguridad CORS, que impide por defecto que un origen realice peticiones a un dominio distinto. Se solucionó implementando la clase CorsConfig en Spring Boot, que configura el servidor para aceptar peticiones desde cualquier origen durante la fase de desarrollo.

8. Conclusións y reflexiones

9. Bibliografía y Webgrafía

10. Anexo I – Manual técnico de instalación o puesta en marcha

11. Anexo II – Documentación de uso (manuales usuario)

12. Anexo III – Otra documentación